ECE

Major: Cybersecurity International

Course
Sécurité des Containers

Lab Report
Session 3

Instructor
Etienne LOUTSCH

Group Members

- ARDILLON Bastian
- SAVEAUX Octave
- BRULEY Martin

Date
February 25, 2026

Activités Pratiques

Déployer un Cluster Kubernetes avec Kind

Nous avons installé Kind et kubectl, puis créé un cluster Kubernetes avec 2 nœuds control-plane et 2 nœuds worker en utilisant un fichier de configuration YAML

```
kind create cluster --config cluster-config.yaml
```

La création du cluster a été effectuée avec succès :

```
octave@PCoctave:~$ sudo kind create cluster --config cluster-config.yaml --name mon-cluster-tp
Creating cluster "mon-cluster-tp" ...
 ✓ Ensuring node image (kindest/node:v1.27.3) 🖼
 ✓ Preparing nodes 📦 📦 📦 📦
 ✓ Configuring the external load balancer ⚖️
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
 ✓ Joining more control-plane nodes 🎮
 ✓ Joining worker nodes 🚜
Set kubectl context to "kind-mon-cluster-tp"
You can now use your cluster with:

kubectl cluster-info --context kind-mon-cluster-tp

Not sure what to do next? 😅  Check out https://kind.sigs.k8s.io/docs/user/quick-start/
```

Figure 1: création du cluster Kubernetes avec Kind

Pour vérifier que le cluster est correctement déployé, nous avons exécuté :

```
kubectl get nodes
```

Le résultat affiche bien les quatre nœuds :

```
octave@PCoctave:~$ sudo kubectl get nodes
NAME                          STATUS   ROLES           AGE     VERSION
mon-cluster-tp-control-plane  Ready    control-plane   5m57s   v1.27.3
mon-cluster-tp-control-plane2 Ready    control-plane   5m39s   v1.27.3
mon-cluster-tp-worker         Ready    <none>          4m48s   v1.27.3
mon-cluster-tp-worker2        Ready    <none>          4m48s   v1.27.3
octave@PCoctave:~$
```

Figure 2: vérification des 4 nœuds du cluster (2 master, 2 workers)

Pour afficher la liste des namespaces du cluster nous avons executé la commande suivante :

```
kubectl get namespaces
```

```
octave@PCoctave:~$ sudo kubectl get namespaces
NAME                   STATUS   AGE
default                Active   9m36s
kube-node-lease        Active   9m36s
kube-public            Active   9m36s
kube-system            Active   9m36s
local-path-storage     Active   9m22s
octave@PCoctave:~$
```

Figure 3: liste des namespaces du cluster

Pour connaître la version de Kubernetes déployée, voici la commande utilisée:

```
kubectl version
```

```
octave@PCoctave:~$ sudo kubectl version
Client Version: v1.35.1
Kustomize Version: v5.7.1
Server Version: v1.27.3
Warning: version difference between client (1.35) and server (1.27) exceeds the supported minor version skew of +/-1
octave@PCoctave:~$
```

Figure 4: version de Kubernetes déployée

La version de Kubernetes déployée est la **v1.27.3** (comme on peut le voir sur les nœuds et dans la sortie de `kubectl version`).

Expérimentation des RBAC

Nous avons créé un namespace dédié à l'expérimentation RBAC se nommant **test-rbac**:

```
kubectl create ns test-rbac
```

Création du namespace test-rbac

Figure 5: création du namespace test-rbac

On vérifie ensuite avec `kubectl get namespaces` que le namespace `test-rbac` apparaît bien dans la liste

```
octave@PCoctave:~$ sudo kubectl get namespaces
NAME                   STATUS   AGE
default                Active   3m37s
kube-node-lease        Active   3m37s
kube-public            Active   3m38s
kube-system            Active   3m38s
local-path-storage     Active   3m24s
test-rbac              Active   11s
octave@PCoctave:~$
```

Figure 5b: vérification de la création du namespace test-rbac

Nous avons déployé un pod nginx dans le namespace **test-rbac** avec un fichier YAML. Voici le fichier YAML utilisé :

```
  GNU nano 6.2                                   mon-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: test-rbac
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

Figure 6: contenu du fichier mon-pod.yaml

Nous avons déployé ce pod avec la commande suivante :

```
kubectl apply -f mon-pod.yaml
```

```
octave@PCoctave:~$ sudo kubectl apply -f mon-pod.yaml
pod/nginx created
octave@PCoctave:~$
```

Figure 7: déploiement du pod nginx dans le namespace test-rbac

Nous avons par la suite afficher les logs de ce pod :

```
kubectl logs nginx -n test-rbac
```

```
octave@PCoctave:~$ sudo kubectl logs nginx -n test-rbac
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/18 14:47:08 [notice] 1#1: using the "epoll" event method
2026/02/18 14:47:08 [notice] 1#1: nginx/1.29.5
2026/02/18 14:47:08 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/18 14:47:08 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/02/18 14:47:08 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/18 14:47:08 [notice] 1#1: start worker processes
2026/02/18 14:47:08 [notice] 1#1: start worker process 32
2026/02/18 14:47:08 [notice] 1#1: start worker process 33
2026/02/18 14:47:08 [notice] 1#1: start worker process 34
2026/02/18 14:47:08 [notice] 1#1: start worker process 35
2026/02/18 14:47:08 [notice] 1#1: start worker process 36
2026/02/18 14:47:08 [notice] 1#1: start worker process 37
2026/02/18 14:47:08 [notice] 1#1: start worker process 38
2026/02/18 14:47:08 [notice] 1#1: start worker process 39
octave@PCoctave:~$
```

Figure 8: logs du pod nginx

Nous avons créé un Role `pod-reader` qui autorise uniquement les actions `get` et `list` sur les pods dans le namespace `test-rbac`, voici le fichier YAML utilisé :

```
  GNU nano 6.2                                  role-pod-reader.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: test-rbac
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

Figure 9: contenu du fichier role-pod-reader.yaml

Pour appliquer ce Role, nous avons executé la commande suivante :

```
kubectl apply -f role-pod-reader.yaml
```

```
octave@PCoctave:~$ nano role-pod-reader.yaml
octave@PCoctave:~$ kubectl apply -f role-pod-reader.yaml
error: error validating "role-pod-reader.yaml": error validating data: failed to download openapi: Get "http://localhost
:8080/openapi/v2?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused; if you choose to ignore these error
s, turn validation off with --validate=false
octave@PCoctave:~$ sudo kubectl apply -f role-pod-reader.yaml
role.rbac.authorization.k8s.io/pod-reader created
octave@PCoctave:~$
```

Figure 10: création du Role pod-reader

Pour afficher ce nouvelle role créé nous avons executé la commande suivante :

```
kubectl get role pod-reader -n test-rbac
```

```
octave@PCoctave:~$ sudo kubectl get role pod-reader -n test-rbac
[sudo] password for octave:
NAME         CREATED AT
pod-reader   2026-02-18T15:10:54Z
octave@PCoctave:~$
```

Figure 11: affichage du Role pod-reader

Nous pouvons bien voir que le Role a été créé avec succès.

Nous avons appliqué un RoleBinding donné dans l'énoncé pour associer le Role `pod-reader` à l'utilisateur `titi`. Voici le fichier YAML utilisé :

```
octave@PCoctave:~$ nano rolebinding-pod-reader.yaml
octave@PCoctave:~$ kubectl apply -f rolebinding-pod-reader.yaml
error: error validating "rolebinding-pod-reader.yaml": error validating data: failed to download openapi: Get "http://lo
calhost:8080/openapi/v2?timeout=32s": dial tcp 127.0.0.1:8080: connect: connection refused; if you choose to ignore thes
e errors, turn validation off with --validate=false
octave@PCoctave:~$ sudo kubectl apply -f rolebinding-pod-reader.yaml
rolebinding.rbac.authorization.k8s.io/read-pods-binding created
octave@PCoctave:~$
```

Figure 12: liaison du Role avec l'utilisateur titi

Pour créer l'utilisateur fictif `titi`, nous avons récupéré la CA du cluster depuis le conteneur Kind :

```
docker cp kind-control-plane:/etc/kubernetes/pki/ca.crt .
docker cp kind-control-plane:/etc/kubernetes/pki/ca.key .
```

Récupération de la CA

Figure 13: récupération de l'autorité de certification du cluster

Ensuite, nous avons généré les certificats pour l'utilisateur titi :

```
openssl genrsa -out titi.key 2048
openssl req -new -key titi.key -out titi.csr -subj "/CN=titi"
openssl x509 -req -in titi.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
titi.crt -days 365
```

```
octave@PCoctave:~$ openssl genrsa -out titi.key 2048
octave@PCoctave:~$ openssl req -new -key titi.key -out titi.csr -subj "/CN=titi"
octave@PCoctave:~$ openssl x509 -req -in titi.csr -CA ca.crt -CAkey ca.key CAcreateserial -out titi.crt -days 365
x509: Use -help for summary.
octave@PCoctave:~$ openssl x509 -req -in titi.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out titi.crt -days 365
Certificate request self-signature ok
subject=CN = titi
Could not open file or uri for loading CA private key from ca.key
806BA23A907B0000:error:16000069:STORE routines:ossl_store_get0_loader_int:unregistered scheme:../crypto/store/store_register.c:237:scheme=fi
le
806BA23A907B0000:error:8000000D:system library:BIO_new_file:Permission denied:../crypto/bio/bss_file.c:67:calling fopen(ca.key, rb)
806BA23A907B0000:error:10080002:BIO routines:BIO_new_file:system lib:../crypto/bio/bss_file.c:77:
octave@PCoctave:~$
```

Figure 14: génération des certificats pour l'utilisateur titi

Puis nous avons ajouté l'utilisateur titi au contexte kubectl et créé un contexte dédié :

```
kubectl config set-credentials titi --client-certificate=titi.crt --client-
key=titi.key
kubectl config set-context titi-context --cluster=kind-kind --namespace=test-rbac
--user=titi
```

```
octave@PCoctave:~$ sudo kubectl config set-credentials titi \
> --client-certificate=titi.crt \
> --client-key=titi.key
User "titi" set.
octave@PCoctave:~$
```

Figure 15: ajout de l'utilisateur titi au contexte kubectl

```
octave@PCoctave:~$ sudo kubectl config set-context titi-context \
> --cluster=kind-kind \
> --namespace=test-rbac \
> --user=titi
Context "titi-context" created.
octave@PCoctave:~$
```

Figure 16: création du contexte titi-context dans le namespace test-rbac

Nous avons vérifié que l'utilisateur était bien pris en compte dans la liste des credentials disponibles.

Vérification presence credentials titi

Figure 17: vérification de la présence titi

Pour utiliser le contexte de l'utilisateur titi nous avons executé la commande suivante :

```
kubectl config use-context titi-context
```

```
octave@PCoctave:~$ kubectl config use-context titi-context
Switched to context "titi-context".
octave@PCoctave:~$ kubectl config current-context
titi-context
octave@PCoctave:~$
```

Figure 18: basculement vers le contexte titi-context

Avec le contexte `titi-context`, nous pouvons lister les pods dans le namespace `test-rbac` car le Role `pod-reader` autorise les verbes `get` et `list` sur les pods :

```
kubectl get pods -n test-rbac
```

```
octave@PCoctave:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          36m
octave@PCoctave:~$
```

Figure 19: accès à la liste des pods par l'utilisateur titi

Le résultat confirme que l'utilisateur titi peut accéder à la liste des pods (le pod nginx apparaît en statut Running).

En revanche, l'utilisateur titi ne dispose pas de la possibilité de créer un pod. Lorsque nous essayons de créer un pod :

```
kubectl run test --image=nginx -n test-rbac
```

```
octave@PCoctave:~$ kubectl run test --image=nginx
Error from server (Forbidden): pods is forbidden: User "titi" cannot create resource "pods" in API group "" in the names
pace "test-rbac"
```

Figure 20: accès refusé — titi ne peut pas créer de pods

Nous obtenons une erreur `Forbidden` : *User "titi" cannot create resource "pods" in API group "" in the namespace "test-rbac"*. Cela démontre que le RBAC restreint correctement les permissions : titi peut lire les pods mais pas en créer.

Scanner un Cluster Kubernetes avec Kube-Bench

Nous avons déployé Kube-Bench en tant que Job Kubernetes pour scanner la sécurité du cluster. Le Job a été appliqué avec :

```
kubectl apply -f job.yml
```

Nous avons vérifié que le Job s'exécute correctement.

```
octave@PCoctave:~$ kubectl apply -f job.yaml
job.batch/kube-bench created
octave@PCoctave:~$ kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
kube-bench-7jwrf    0/1     ContainerCreating   0          7s
octave@PCoctave:~$
```

Figure 21: application du Job Kube-Bench et vérification de l'exécution

Le scan Kube-Bench analyse le cluster selon les recommandations CIS (Center for Internet Security) et produit un rapport listant les contrôles passés, avertis et échoués. Les résultats permettent d'identifier les configurations à durcir

```
== Summary policies ==
0 checks PASS
0 checks FAIL
35 checks WARN
0 checks INFO

== Summary total ==
11 checks PASS
6 checks FAIL
42 checks WARN
0 checks INFO

octave@PCoctave:~$
```

Figure 22: résultats du scan Kube-Bench

Kube-Bench a détecté 11 PASS, 6 FAIL et 42 WARN. Les FAIL sont à corriger, beaucoup de WARN indiquent des points à améliorer. Notre cluster est donc pas optimisé

Détection et alerte d'intrusions avec Falco

Nous avons ajouté le dépôt Helm Falco et installé la chart dans un namespace dédié :

```
octave@PCoctave:~$ helm repo add falcosecurity https://falcosecurity.github.io/charts
"falcosecurity" has been added to your repositories
octave@PCoctave:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "falcosecurity" chart repository
Update Complete. ⎈Happy Helming!⎈
octave@PCoctave:~$ kubectl create ns falco
namespace/falco created
octave@PCoctave:~$ helm -n falco install falco falcosecurity/falco --set falcosidekick.enabled=true --set falcosidekick.
webui.enabled=true
NAME: falco
LAST DEPLOYED: Wed Feb 25 15:44:27 2026
NAMESPACE: falco
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
Falco agents are spinning up on each node in your cluster. After a few
seconds, they are going to start monitoring your containers looking for
security issues.
octave@PCoctave:~$
```

Figure 23: ajout du dépôt Helm et installation de Falco

Après quelques minutes, nous avons vérifié que les pods du namespace `falco` sont en état Running :

```
kubectl get pods -n falco
```

```
octave@PCoctave:~$ kubectl get pods -n falco
NAME                                      READY   STATUS    RESTARTS   AGE
falco-4bhn8                               2/2     Running   0          6m24s
falco-525kr                               2/2     Running   0          6m24s
falco-bt6zb                               1/2     Running   0          6m24s
falco-falcosidekick-5f4bbb544b-c7mns      1/1     Running   0          6m24s
falco-falcosidekick-5f4bbb544b-st455      1/1     Running   0          6m24s
falco-falcosidekick-ui-5c97b4c869-pj2kb   1/1     Running   0          6m24s
falco-falcosidekick-ui-5c97b4c869-xsjb7   1/1     Running   0          6m24s
falco-falcosidekick-ui-redis-0            1/1     Running   0          6m24s
falco-w2bg7                               2/2     Running   0          6m24s
octave@PCoctave:~$
```

Figure 24: vérification des pods Falco en Running

Tous les pods sont en état *Running*, l'installation s'est parfaitement déroulée et tout fonctionne comme attendu.

Pour afficher l'interface web de Falco, nous avons effectué un port-forward dans notre navigateur :

```
kubectl port-forward svc/falco-falcosidekick-ui 2802:2802 --namespace falco
```

```
octave@PCoctave:~$ kubectl port-forward svc/falco-falcosidekick-ui 2802:2802 -n falco
Forwarding from 127.0.0.1:2802 -> 2802
Forwarding from [::1]:2802 -> 2802
Handling connection for 2802
Handling connection for 2802
Handling connection for 2802
Handling connection for 2802
Handling connection for 2802
```

Figure 25: commande port-forward pour l'interface Falco

Puis ouvert l'URL http://127.0.0.1:2802 dans le navigateur.

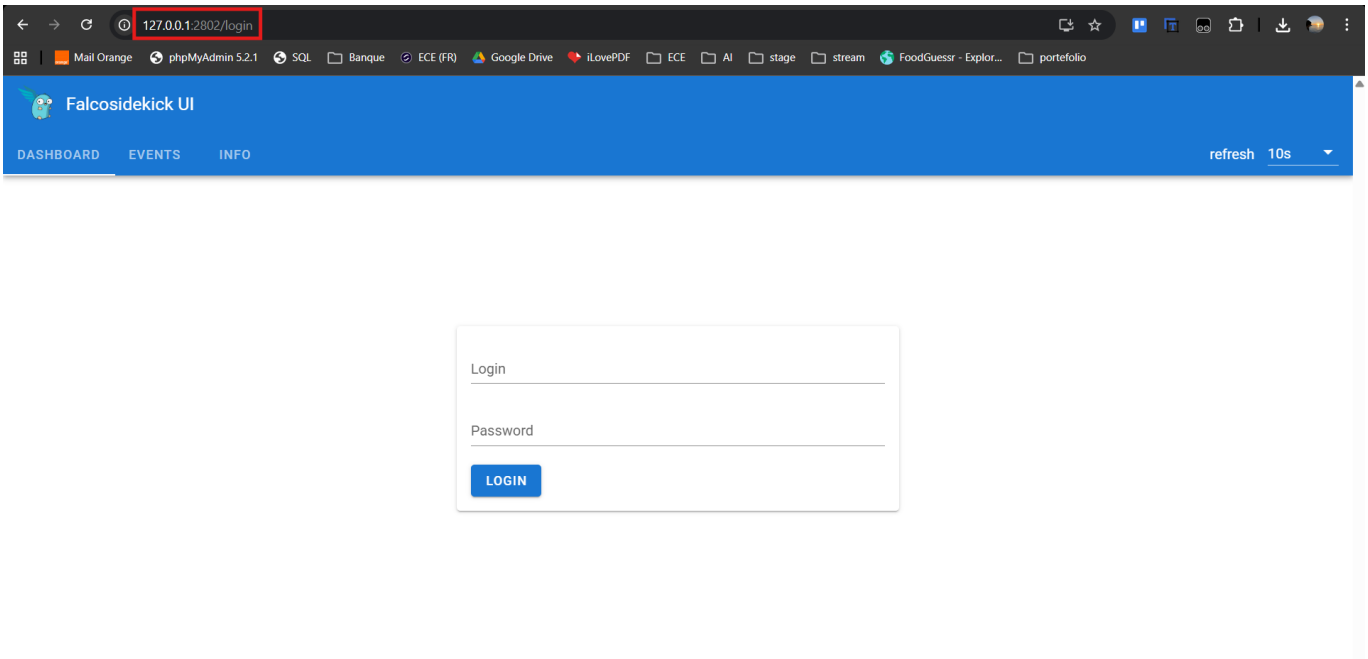

Figure 26: interface Falco dans le navigateur

Nous avons déployé un pod basé sur Alpine avec un fichier YAML donné dans l'énoncé :

```
octave@PCoctave:~$ nano mon-pod.yml
octave@PCoctave:~$ kubectl apply -f mon-pod.yml
pod/front created
octave@PCoctave:~$
```

Figure 26: création et déploiement du pod front

Puis nous avons exécuté un shell dans le pod et lu le fichier `/etc/shadow`. Falco a détecté cette action et généré une alerte :

Timestamp	Source	Hostname	Priority	Rule	Output	Tags
2026/02/25 16:30:48:099	syscall	mon-cluster-tp-worker	Warning	Read sensitive file untrusted	15:30:48.099647033: Warning Sensitive file opened for reading by non-trusted program | file=/etc/shadow gparent=containerd-shim ggparent=<NA> gggparent=<NA> evt_type=open user=root user_uid=0 user_loginuid=-1 process=cat proc_exepath=/bin/busybox parent=sh command=cat /etc/shadow terminal=34816 container_id=96175d3e29dd container_name=front container_image_repository=docker.io/library/alpine container_image_tag=latest k8s_pod_name=front k8s_ns_name=default container.id 96175d3e29dd container.image.repository docker.io/library/alpine container.image.tag latest container.name front evt.time 1772033448099647000 evt.type open fd.name /etc/shadow k8s.ns.name default k8s.pod.name front proc.aname[2] containerd-shim proc.aname[3] proc.aname[4] proc.cmdline cat /etc/shadow proc.exepath /bin/busybox proc.name cat proc.pname sh proc.tty 34816 user.loginuid -1 user.name root user.uid 0	T1555 container filesystem host maturity_stable mitre_credential_access

Figure 27: alerte Falco — lecture du fichier /etc/shadow (Priorité: Warning, Règle: Read sensitive file untrusted)

Toujours dans le shell du pod `front`, nous avons effectué une requête vers l'API Kubernetes :

Falco peut également détecter ce type d'activité. L'alerte générée a pour priorité **Warning** et concerne la règle liée aux connexions sortantes vers l'API Kubernetes.

Alerte Falco - Requête API Kubernetes

Figure 28: alerte Falco — requête vers l'API Kubernetes depuis le pod